Research Statement

Artem Pelenitsyn

High-level programming languages power today's AI and data-science ecosystems. They let teams move fast, iterate, and share results—but the workloads they drive are compute-hungry, memory-intensive, and unforgiving of inefficiency. When models and datasets scale, a penalty from dynamic dispatch, boxing, or poor memory locality translates directly into higher costs and longer time-to-insight. At the same time, these stacks are error-prone: subtle type-mismatch issues, misaligned data layouts, or unsafe optimizations can silently corrupt results or derail performance. But practitioners should not choose between productivity and performance.

My research aims to make high-level programming both fast and predictable. I combine formal methods from programming languages with algorithmic and hardware-aware techniques to address opaque performance cliffs in dynamic languages, inefficient memory layouts for tree-structured data, and missed opportunities to use specialized GPU units for irregular workloads. Program semantics and logical calculi provide precise definitions and proof obligations for the optimizations we want, while systems-level design delivers speedups on real workloads.

1 Background and Experience

To achieve fast and predictable high-level programming, I have been developing principled analyses, robust memory representations, and hardware mappings following three complementary research directions.

- Julia: I paired formal models with at-scale measurement to pin down the key optimization property of type stability—i.e., when concrete types can be inferred—and quantify its impact.
- Gibbon: I advanced data layout and memory management for tree-structured data to accelerate traversal-heavy workloads.
- Ray-tracing (RT) cores: I generalized fixed-function ray-tracing units to non-graphics algorithms (e.g., non-Euclidean k-NN, Barnes–Hut), mapping irregular computations onto specialized GPU hardware.

My contributions span first-author leadership and late-stage turnarounds—co-designing formalisms, instrumenting runtimes, architecting evaluations, and restructuring papers to sharpen claims and ensure reproducibility.

1.1 Julia: performance and dynamism for scientific computing

Background: Julia and the two-language problem. Julia is a language popular in scientific and data-intensive computing and was created to mitigate the "two-language problem," where teams prototype in a high-level dynamic language and then re-implement performance-critical kernels in C/C++ or Fortran. Julia aims to unify productivity and speed via multiple dispatch over rich parametric types, aggressive method specialization, and an LLVM-based JIT. A pivotal concept in this model is type stability: when result types can be inferred independent of runtime data, the compiler can specialize methods, avoid dynamic dispatch and boxing, and unlock downstream

optimizations; when it cannot, execution falls back to dynamic paths that can cause unpredictable slowdowns. These predictability issues—and the absence of a precise, language-level account of type stability—can be a barrier to wider adoption in performance-critical settings. Prior to our work, type stability and its empirical effects were largely informal and tool-driven rather than precisely defined and systematically evaluated.

Result: formalizing and evaluating type stability in Julia [5]. I co-designed a precise account of type stability within Julia's compilation model and led the empirical evaluation pipeline end-to-end. Concretely, I built the data collection and analysis tooling, assembled real-world Julia workloads, and measured how type stability affects specialization opportunities and optimization outcomes in practice. This study clarified when and why Julia achieves its performance promise without sacrificing dynamic expressiveness, surfaced common sources of instability in idiomatic code, and provided actionable guidance to developers and compiler engineers. For AI/DS, this work makes performance regressions detectable in CI and reduces cloud spend by eliminating avoidable dynamic paths. I led the project and co-designed the formalism; I solely performed data collection and empirical analysis, which got a special recognition from the reviewers.

1.2 Gibbon: a tree-traversal compiler

Background: inefficient representation of datatypes in high-level languages. High-level languages typically represent algebraic data types (ADTs) such as trees using pointer-rich, boxed layouts managed by tracing garbage collectors. While flexible, this default often performs poorly for traversal-heavy workloads: indirection hurts cache locality and branch prediction, boxing inflates memory footprint, and frequent allocation and tracing increase GC pressure. Two levers interact here—data layout (boxed vs. packed/contiguous variants; tag encodings; field ordering) and memory management (pure tracing GC vs. regions/arenas)—but today these choices are largely implicit, manual, and disconnected from the structure of traversals. The Gibbon project explores compiler- and runtime-level techniques that specialize both layout and allocation to tree traversals, aiming to reduce indirection, improve spatial locality, and align allocation lifetimes with traversal phases. This enables fast tree/graph analytics in functional languages without rewriting kernels in low-level languages such as C and C++.

Result: combining region-based and tracing garbage collection for datatypes [1]. Pure region-based allocation aligns well with traversal phases but struggles with long-lived data and irregular lifetimes; pure tracing GC handles general lifetimes but imposes overheads that can dominate in allocation-heavy traversals. This work proposes a hybrid strategy: use regions (or arenas) for short-lived traversal artifacts and tracing GC for long-lived objects, with compiler/runtime cooperation to route allocations and trigger reclamation at traversal boundaries. A key concern in earlier reviews was limited evaluation scope. I formulated a broader evaluation plan—adding workloads, stress tests, and comparative baselines—to address those critiques and contributed analysis of the results to pinpoint when and why the hybrid wins. The outcome is a clearer empirical picture of the conditions under which hybrid management outperforms either technique alone and guidance for integrating such hybrids into high-level language runtimes.

Result: automated decisions on memory layout of datatypes [6]. This work studies how to automatically select ADT layouts that balance locality, space usage, and interoperability with the host runtime. The system explores layout alternatives guided by an explicit model of traversal behavior, then generates code and data representations accordingly. Our evaluation examines traversal-intensive benchmarks and real workloads to identify when contiguous, packed encodings

dominate (due to fewer cache misses and reduced GC traffic) and when boxed layouts remain preferable (e.g., for mutation patterns or interop constraints). I joined the project after a paper rejection and helped sharpen the problem statement and claims, restructure the exposition (especially the opening sections) to connect the model to empirical findings. I also re-designed the evaluation protocol (workload selection, ablations, and reporting) that makes the trade-offs legible to both compiler and systems audiences.

1.3 Ray-tracing cores: generalized GPU compute

Background: ray-tracing hardware for non-graphics tasks. Modern GPUs include small dedicated units for ray tracing—hardware that follows a line through a scene and very quickly decides what it hits. These units traverse a tree of bounding boxes to skip most objects, which is essentially a fast hierarchical search. Many non-graphics problems have the same shape: search a spatial or tree-structured dataset and prune most candidates early (for example, neighbor queries, collision detection, or sparse simulation updates). The idea is to store our data in a similar tree and express queries as "ray" traversals so we can reuse this hardware's high throughput and energy efficiency. This opens specialized silicon to irregular workloads common in simulation and data analysis. Two challenges recur: (1) the hardware interface is constrained, so we must reformulate non-graphics goals to fit what these units can control; and (2) correctness and performance are subtle when our acceptance criteria are not the Euclidean geometry the hardware assumes—we need to show that our pruning remains safe and that speedups are predictable.

Result: nearest-neighbors search in non-Euclidean distance [2]. Prior reductions targeted Euclidean k-Nearest Neighbors search (k-NN) by leveraging geometric bounds native to graphics pipelines. Our work reframes the problem: we develop a standalone framework for k-NN on ray-tracing hardware that applies to a wide class of metric distances, separating the algorithmic requirements (safe pruning via metric bounds) from hardware-specific traversal mechanics. Conceptually, we show how to encode conservative bounds and pruning decisions within RT traversal so that correctness is preserved for non-Euclidean metrics, and we outline how to specialize the encoding to concrete metrics as needed. I pivoted the paper from a narrow generalization of an existing reduction to a coherent, metric-agnostic framework; I restructured the narrative, clarified the invariants that guarantee correctness, and aligned the claims with the evaluation; finally, I found several bugs in the formalism and fixed them. The final version received a best paper award.

Result: ray-tracing-based Barnes—Hut simulation [3]. Barnes—Hut accelerates N-body simulation via hierarchical aggregation with an acceptance criterion that trades accuracy for speed. We show how to map this computation to RT hardware by using acceleration structures to organize bodies hierarchically and by expressing force-accumulation queries as ray operations that traverse and prune according to the acceptance rule. The key is to exploit the hardware's fast culling while maintaining control over approximation error. I contributed at the evaluation and study-design level: I proposed the protocol, selected and balanced synthetic and real-world datasets, and ensured comparative baselines that stress distinct regimes (e.g., clustered vs. uniform distributions). I also helped shape the exposition so that the performance-accuracy trade-offs and the conditions under which the RT-based approach outperforms conventional GPU compute are clear and reproducible.

2 Research Plan

I propose practical solutions towards performant, predictable, and portable high-level programming. My three complementary thrusts are developer-facing analyses and policies for dynamic languages (Julia), compiler and foundational work to bring traversal-optimized packed data representations into general-purpose ecosystems (Gibbon/GHC), and principled use of specialized silicon (RT cores) via general mappings and a high-level DSL. The unifying goal is to preserve expressiveness while delivering systems-level efficiency and measurable end-to-end wins. Each project is scoped to paper-sized milestones with open-source artifacts, benchmarks, and evaluations on real workloads.

2.1 Scientific computing accelerated (Julia)

Project 1. Robust type-stable code In the era of massive datasets, performance in scientific and data-centric languages such as Julia or Python is vital. My prior work [5] shed light on Julia's compiler-based optimization techniques and showed how much the language ecosystem depends on them. A logical follow-up is to make users aware of code properties like type stability and help them keep it in check. To this end, I have proposed [4] presents a framework to check type stability in Julia, but it does not define a precise method and instead stops at a crude brute-force approach. I plan to extend this work and develop a method that accounts for imprecise types in Julia using source-code analysis. In particular, the new method will process Julia functions with type-annotated parameters as before [4], but for unannotated parameters it will define an analysis of the use sites of each parameter inside the method body to narrow the list of possible types for that parameter. This new method of checking type stability will fit into my earlier framework [4].

Project 2. Efficient memory management for data science One of the major sources of slowdown for high-level languages like Julia or Python is inefficient memory-management patterns, such as many small allocations inside hot loops. A recent development integrated the state-of-the-art research garbage collector MMTk into Julia [7], demonstrating the feasibility of pluggable memory management for garbage-collected languages. However, tracing garbage collection is not the only way to manage memory: another approach is deterministic, scope-based reclamation based on lexical allocation scopes. For instance, the long line of work on region-based memory management and ownership types underpins the success of the Rust programming language with its lifetime tracking and borrow checking. Our previous work on Gibbon [1] shows how to combine region-based memory management with a traditional tracing, generational garbage collector. This experience will inform our work on a similar experiment for Julia and possibly other languages.

2.2 Versatile and generic tree-traversals accelerator (Gibbon)

Project 1. Portable tree-traversal accelerator for the GHC compiler

Our prior work on Gibbon showed its strong performance on a particular class of algorithms we call tree traversals. But realistic applications usually consist of varying workloads that may or may not map well onto Gibbon's strengths. A promising way to address this concern is to embed Gibbon into a general-purpose system that performs well on mixed workloads and can offload tree traversals to Gibbon. A natural candidate is the GHC Haskell compiler, because Gibbon's input language is close to Haskell.

The first subproject in this space would be the design and implementation of the combined GHC–Gibbon system. The proposed combination of compilers will require modifications to both GHC

and Gibbon. Central to the design will be a system of user-provided annotations in a GHC/Haskell program. These annotations will mark the program fragments that should be compiled with Gibbon. Essential technical work—extracting relevant parts of the GHC Core IR corresponding to the annotated fragments and compiling them with Gibbon—has been done before (but not published yet). What remains for the first publication is nailing down the embedding technique: currently, Gibbon's setup is stateful with respect to the file system; it has to find its runtime and standard library and be able to run a C compiler. All these aspects need coordination with GHC counterparts, which will be informed by my prior contributions to GHC.

A second subproject in this topic is high-performance Haskell libraries. While GHC's optimization engine is powerful, achieving top performance also relies on implementation techniques encoded in libraries (type classes, type families, rewrite rules, etc.). For instance, most HPC-related code in Haskell is based on the `vector` library, which makes heavy use of data families (a variant of type families); hence Gibbon needs to understand how to employ `vector`-native structures with its own array type. This integration will require abstraction over a particular array representation in Gibbon, which, in turn, requires adaptation of the Gibbon formalism [8]. Our prior work on making datatype representation in Gibbon more flexible [6] will inform work on this subproject.

Project 2. Foundations and new backends for tree-traversals on packed data Optimizing compilers are error-prone, and to mitigate this, compilers such as GHC and Gibbon employ underlying logical calculi that embody a representative subset of the input language and are proven type-safe. Gibbon's underlying calculus, LoCal, was proven type-safe but is tied to a particular packed representation of datatypes used in Gibbon. This creates challenges when we generalize Gibbon to use different backends with different packed representations—for example, switching between array types (see Project 1 above), supporting different evaluation orders (Project 1), changing how datatypes are serialized [6], or employing memory-layout optimizations like switching between array-of-structures and structure-of-arrays (in the works). Beyond the technical challenges of extending LoCal to these applications, such extensions are also error-prone because LoCal has never been mechanized to have its soundness checked automatically by a proof assistant. A particularly useful extension, therefore, is to mechanize LoCal and, in the process, abstract it over implementation details that tie it to the original Gibbon [9]. This work will be informed by my prior work on formalization of the Julia JIT [5], which addressed similar challenges.

2.3 A new programmable accelerator (ray-tracing cores)

Project 1. Domain-specific language on top of RT cores Designing new RT-based reductions and applying existing ones is cumbersome because current APIs are low-level, which slows innovation and makes development error-prone. I propose to develop a high-level interface—a domain-specific language (DSL)—that makes it easier to apply existing reductions in new settings. The main goal is to let users work at the level of their problem domain rather than in hardware-level primitives (rays, scenes, intersection callbacks, etc.) exposed by today's RT APIs. The DSL will also handle common, cross-cutting concerns that are currently solved ad hoc by users: reusing results and state across calls to RT hardware, and managing hardware limits such as memory and precision. In my other two research directions, I focus on ensuring performance for high-level tools; here, I take the complementary approach of starting from a high-performance tool (RT cores) and building a high-level abstraction around it. Ensuring that this abstraction preserves the performance benefits of RT cores will be a central task and will proceed in parallel with the other two research directions.

Project 2. More general tree traversals with RT cores

Prior works (including ours [2]) exploited the similarity between ray tracing and nearest-neighbor search (NNS). In contrast, our RT-core–powered version of Barnes—Hut (BH) simulation showed how to map a tree traversal unrelated to NNS onto RT hardware [3]. The uniqueness of this RT-core mapping invites at least two follow-ups.

The first subproject improves the way we model the BH traversal by exposing additional opportunities for parallelism. At present, when computing forces for one body, every node in that body's tree is visited in order, and parallel speedups come primarily from processing many bodies concurrently. A potentially more balanced workload can be achieved if the traversal itself proceeds in parallel: instead of waiting to check the BH cut-off condition, we can speculatively start both possibilities (recursing into subtrees or jumping through the autorope) and later select which result to keep. Given the massive parallelism that RT cores provide, this finer-grained parallel approach appears beneficial.

The second subproject will try to accommodate spatial-tree traversals similar to BH—for example, treecodes and the Fast Multipole Method. In particular, a large family of hierarchical algorithms accelerates "all-pairs" problems where a spatial hierarchy plus a distance-based admissibility test (akin to the BH cut-off condition) turns the $O(n^2)$ complexity into $O(n \log n)$. Our RT-core-backed traversal strategy for BH should carry over to these algorithms with minimal changes.

References

[1] Chaitanya S. Koparkar, Vidush Singhal, Aditya Gupta, Mike Rainey, Michael Vollmer, **Artem Pelenitsyn**, Sam Tobin-Hochstadt, Milind Kulkarni, and Ryan R. Newton. "Garbage Collection for Mostly Serialized Heaps". In: *Proceedings of the 2024 ACM SIGPLAN International Symposium on Memory Management, ISMM 2024, Copenhagen, Denmark, 25 June 2024*. Ed. by Michael D. Bond, Jae W. Lee, and Hannes Payer. ACM, 2024, pp. 1–14. DOI: 10.1145/3652024.3665512.

[2] Durga Keerthi Mandarapu, Vani Nagarajan, **Artem Pelenitsyn**, and Milind Kulkarni. "Arkade: k-Nearest Neighbor Search With Non-Euclidean Distances using GPU Ray Tracing". In: *Proceedings of the 38th ACM International Conference on Supercomputing, ICS, Kyoto, Japan*. Ed. by Kenji Kise, Valentina Salapura, Murali Annavaram, and Ana Lucia Varbanescu. ACM, 2024, pp. 14–25. DOI: 10.1145/3650200.3656601.

[3] Vani Nagarajan, Rohan Gangaraju, Kirshanthan Sundararajah, **Artem Pelenitsyn**, and Milind Kulkarni. "RT-BarnesHut: Accelerating Barnes-Hut Using Ray-Tracing Hardware". In: *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, PPoPP 2025, Las Vegas, NV, USA, March 1-5, 2025*. ACM, 2025, pp. 43–56. DOI: 10.1145/3710848.3710885.

[4] **Artem Pelenitsyn**. "Approximating Type Stability in the Julia JIT (Work in Progress)". In: *Proceedings of the 15th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, VMIL 2023, Cascais, Portugal, 23 October 2023*. Ed. by Andrea Rosà and Martin Henz. ACM, 2023, pp. 83–87. DOI: 10.1145/3623507.3623556.

[5] **Artem Pelenitsyn**, Julia Belyakova, Benjamin Chung, Ross Tate, and Jan Vitek. "Type Stability in Julia: Avoiding Performance Pathologies In JIT compilation". In: *Proc. ACM Program. Lang.* 5.OOPSLA (2021), pp. 1–26. DOI: 10.1145/3485527.

[6] Vidush Singhal, Chaitanya Koparkar, Joseph Zullo, **Artem Pelenitsyn**, Michael Vollmer, Mike Rainey, Ryan Newton, and Milind Kulkarni. "Optimizing Layout of Recursive Datatypes with Marmoset: Or, Algorithms + Data Layouts = Efficient Programs". In: *38th European Conference on Object-Oriented Programming, ECOOP 2024, September 16-20, 2024, Vienna, Austria*. Ed. by Jonathan Aldrich and Guido Salvaneschi. Vol. 313. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, 38:1–38:28. DOI: 10.4230/LIPICS.ECOOP.2024.38.

[7] Luis Eduardo de Souza Amorim, Yi Lin, Stephen M. Blackburn, Diogo Netto, Gabriel Baraldi, Nathan Daly, Antony L. Hosking, Kiran Pamnany, and Oscar Smith. “Reconsidering Garbage Collection in Julia: A Practitioner Report”. In: *Proceedings of the 2025 ACM SIGPLAN International Symposium on Memory Management*. ISMM. 2025, pp. 72–83. ISBN: 9798400716102. DOI: 10.1145/3735950.3735957.

[8] Michael Vollmer, Chaitanya Koparkar, Mike Rainey, Laith Sakka, Milind Kulkarni, and Ryan R. Newton. “LoCal: A Language for Programs Operating on Serialized Data”. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI. 2019, pp. 48–62. DOI: 10.1145/3314221.3314631.

[9] Michael Vollmer, Sarah Spall, Buddhika Chamith, Laith Sakka, Chaitanya Koparkar, Milind Kulkarni, Sam Tobin-Hochstadt, and Ryan R. Newton. “Compiling Tree Transforms to Operate on Packed Representations”. In: *31st European Conference on Object-Oriented Programming (ECOOP 2017)*. Ed. by Peter Müller. Vol. 74. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017, 26:1–26:29. DOI: 10.4230/LIPIcs.ECOOP.2017.26.